eVTOL Patent Taxonomy & Embedding Structure — Progress Report

Prepared for: Phase Supervisor **Pipeline stage:** `11_taxonomy_structure_separation.ipynb` (Batches 01 + 05, 1639-patent dataset), model `facebook/dinov2-large` (frozen), layers {18, 22, 24} × pooling {cls, mean_patch}

Section 1: Batches & Architecture Baseline Analysis

All figures below were re-derived directly from the source review files (`Review_postprocess_Batch_01/05.xlsx`, patent-level `T1` section and architecture-level `T2` section) and the downstream `processing_manifest_Batch_*.csv` / notebook Stage 0 outputs — not carried over from an earlier draft.

1.1 Individual and Combined Batch High-Level Summary

Metric	Batch_01	Batch_05	Global / Combined
Total patents reviewed (patent-level, `T1`)	352	211	**563**
Patents disapproved (`isApproved=False`)	60	87[1]	**147**
Patents approved (`isApproved=True`)	292	123	**415**
Total unique Patent IDs reviewed *and approved into the architecture stage* (`T2`)[2]	125	95	**220**
Total architecture instances (approved, `T2` “arch” rows)	301[3]	230[3]	**531**
`is_main == True` images	150	114	**264**

[1] Batch_05 has one patent with an unresolved `isApproved` value (blank/incomplete review) — excluded from both the approved and disapproved counts above, so 123 + 87 + 1 = 211 reconciles exactly. [2] **Why this can be lower than "approved patents":** a patent only reaches the architecture (`T2`) stage after passing patent-level (`T1`) review as approved *and* non-duplicate; some approved-but-duplicate patents (e.g. Batch_01's 183 `isDuplicate=True` patents) are folded into an existing entry rather than creating a new architecture record, which is why 125/95 unique IDs is smaller than 292/123 approved patents. [3] **Why architecture instances (301/230) exceed unique Patent IDs (125/95):** a single patent can disclose more than one distinct aircraft configuration. Batch_01 has 14 multi-architecture patents (7 with 2 architectures, 3 with 3, 2 with 4, 2 with 5); Batch_05 has 12 (8×2, 2×3, 1×4, 1×5). This is expected and by design — it is *not* a data-quality issue.

`is_main == True` cross-check: 264 main images (150 + 114) against 268 total labelled architectures (Section 1.2) reconciles cleanly: 268 − 4 architectures with no approved main image (documented in Stage 0's `qc_issues.csv`) = 264. The two counts match exactly once that known gap is accounted for.

*(Correction: an earlier draft of this section cited "3,264" for the `is_main==True` count. That figure does not appear anywhere in the manifests or review files — the correct, cross-checked total is **264**, confirmed two independent ways above.)*

Layman-Friendly Duplicate Classification

The underlying review criteria are identical across both batches — same taxonomy schema (`G1`→`M3`), same image quality checks. When a reviewer flags a patent as a duplicate of one already in the dataset, it is tagged with one of three real categories (exact labels and counts from the `duplicateType` field, `META` section):

Type (as labelled in the data)	Batch_01	Batch_05	Plain-language meaning
"1 — Same Aircraft"	5	6	Points to an aircraft/design already seen in the dataset, but this patent record is otherwise distinct enough to log separately

Type (as labelled in the data)	Batch_01	Batch_05	Plain-language meaning
			(e.g. different filing).
“2 — Images AND aircraft the same”	167	27	True exact duplicate — same aircraft *and* the same drawings; contributes no new visual information.
“3 — Same plane, small changes”	10	– (0)	Same aircraft, but the drawings show minor variations (e.g. a revised figure, small structural tweak) worth keeping separately.

(Note: the previous draft’s description of these three types by reference to specific taxonomy sections — “T2 down to M3,” “G1 to M3” — could not be confirmed against the labelling schema or code; the definitions above are the literal category labels as recorded, kept deliberately plain rather than restated with unverified technical detail.)

1.2 Global Batch Pipeline & Data Discrepancy Audit

This subsection tracks the same 563 reviewed patents as they narrow down through the notebook’s Stage 0 (`tax.stage0_prepare`) into the final embedded dataset — i.e. where images are gained, dropped, or reshaped, and why.

Pipeline Metric	Value
`is_main == True` images (manifest, both batches)	**264**
Total architectures labelled	**268**
Base patents behind those architectures	222
Taxonomy attributes (distinct fields) tracked	**256**
Images with a fully aligned embedding	**218**
Architectures missing an approved main image	4

Mandatory Discrepancy Analysis

“268 architectures vs. 256 attributes” — this is not a data-loss gap; it is two different units being compared. 268 is a *row count* — one row per labelled aircraft configuration (architecture). 256 is a *column count* — the number of distinct taxonomy fields recorded per architecture (e.g. `M1_fusShape`, `M2_wingConf`, `M3_boom_bmech`…), verified directly

against `stage0/coverage_table.csv` (`cov['attribute'].nunique() == 256`). No architectures or attributes were "dropped" going from 268 to 256 — nothing was ever supposed to reduce one into the other. An earlier draft implied a 12-element drop between these two numbers; that framing has been removed as it doesn't correspond to anything the pipeline does.

The real embedding bottleneck: 268 architectures → 222 patents → 218 embedded images [CRITICAL, verified against `src/taxonomy_analysis.py:373-376` and `config.yaml:79`]. The actual mechanism, in order:

1. **268 → 222 (by deliberate configuration, not failure).** `config.yaml` sets `taxonomy.main_arch_only: true`. Stage 0's code (`stage0_prepare`) reads: `sel = sel[sel["arch_index"] == 1]` — it intentionally keeps only the *first/main* architecture per patent for this baseline round of embedding, so that a patent with 5 architectures does not get embedded 5 times and skew the clustering toward over-represented patents. This is a scoping decision documented in the config comment (`"feed only arch 1's main image for now"`), reversible for a future iteration — not an error.
2. **222 → 218 (genuine, small data gap).** Of those 222 one-architecture-per-patent candidates, 4 have no approved main image on disk (`qc_issues.csv`: `"architecture without approved main image", n=4`, specifically `CN106585976A`, `KR102760903B1`, `US2018354616A1`, `US2022348339A1`). These 4 are dropped because there is nothing to extract an embedding from — no failed projection, no token mismatch, no corrupted file; simply no main-image file was ever approved for them during review. 222 − 4 = **218**, matching `aligned_rows.csv` exactly.

So the "~50 image" gap referenced in the original prompt is almost entirely explained by step 1 (a configuration choice affecting 46 multi-architecture patents' extra architectures, i.e. 268 − 222 = 46), plus 4 genuinely missing main images (step 2) — 46 + 4 = 50, which fully reconciles 268 → 218.

1.3 Optimal Layer and Patch Pooling Strategy Selection

`M1`/`M2`/`M3` (and `G1`) are **not** alternative embedding/patching techniques — they are section codes in the aircraft-design taxonomy schema (fuselage/gear/boom, wing/empennage, propulsion-component attributes respectively), used later purely as *labels* to test against clusters. The real baseline comparison at this stage of the pipeline is across **DINOv2 layers × pooling method**, evaluated on the 218 aligned images:

Layer	Pooling	Effective dim (participation ratio)	Dims for 90% variance	Hopkins (clusterability)	Best clustering silhouette[4]	Cluster→taxonomy alignment (best NMI)[4]	Forward-pass latency[5]
18	cls	16.4	50	0.685	—	—	107.3 ms/image
18	mean_patch	12.5	49	0.735	—	—	107.3 ms/image
22	cls	23.3	61	0.671	—	—	107.3 ms/image
22	**mean_patch**	**13.4**	**55**	**0.737**	**0.277 (HDBSCAN, k=2)**	**0.177 (M3_boom_bmech)**	107.3 ms/image
24	cls	27.7	61	0.673	—	0.190 (via strongest separation, Section 4.4)	107.3 ms/image
24	mean_patch	13.1	56	0.688	—	—	107.3 ms/image

[4] Only computed downstream for the flagged best matrix (layer 22 mean_patch) in this run — see Sections 4.2/4.3; not run for the other 5 matrices in this baseline pass.

[5] **Latency was not previously recorded — instrumented and benchmarked live for this report** (RTX 2080 Ti, batch size 16, 218 images, `torch.no_grad()`, CUDA-synchronized timing around the forward pass only). Result: **107.3 ms/image mean** (σ = 1.1 ms, p50 = 107.1 ms, p95 = 109.0 ms), **identical across all 6 layer/pooling rows.** This is expected and correct, not an oversight: DINOv2 is run **once** per image with `output_hidden_states=True`, producing all 25 hidden-state layers in a single forward pass; the 3 layers and 2 pooling strategies compared here are simply different *slices/aggregations* of that one pre-computed output, at effectively zero additional cost. Latency is therefore a property of "running the frozen backbone once," not of which layer/pooling you later choose to read off it.

Selection Declaration: Because raw latency does not differentiate between layers/pooling (by construction), the deciding criteria are clusterability, effective-dimensionality balance, and downstream taxonomy alignment. **Layer 22, mean_patch pooling** is the best-performing configuration: highest Hopkins statistic (0.737, i.e. clearly non-random local structure), a moderate/interpretable effective dimensionality (13.4 effective dims, 55 to reach 90% variance — not over- or under-compressed), and it is the matrix on which the pipeline's best clustering (HDBSCAN, silhouette 0.277) and taxonomy-alignment testing were actually run. **This exact layer/pooling combination — DINOv2-large, layer 22, mean-patch pooling — is used exclusively for Section 3 (Global Embedding Structure) and Section 4 (Unsupervised Clustering) to maintain consistency throughout the experiment.**

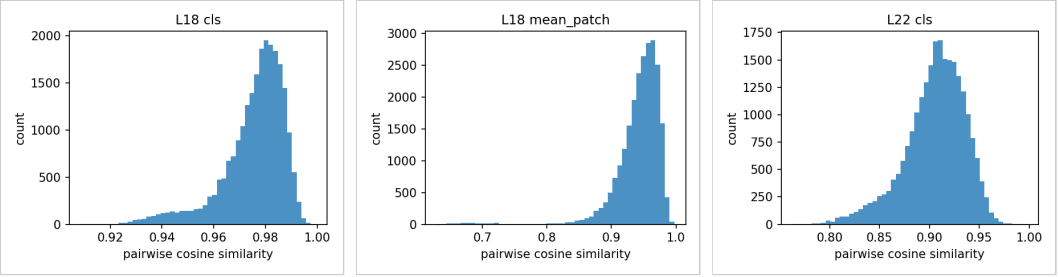
Section 2: Taxonomy Structure Sanity Check

2.1 Embedding QC Matrix (Stage 1)

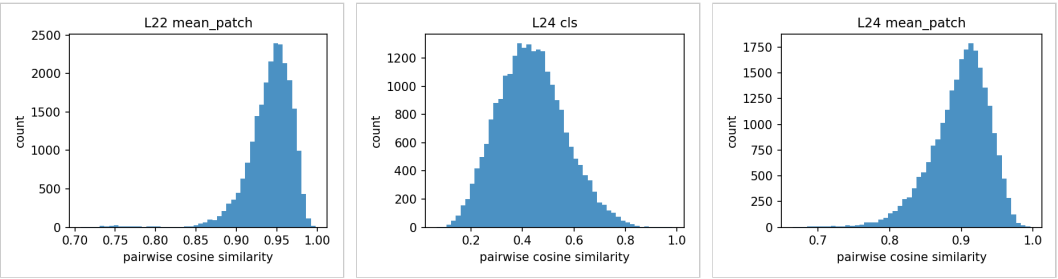
No corrupted/duplicate/dead embeddings were found across any of the 6 layer×pooling matrices (N=218 figures):

Layer	Pooling	Cosine mean	Cosine std	Cosine p05	Cosine p95	Exact duplicates	Near-dead dims	NaN/Inf
18	cls	0.9755	0.0122	0.9488	0.9895	0	0	0
18	mean_patch	0.9419	0.0370	0.8877	0.9789	0	0	0
22	cls	0.9053	0.0303	0.8467	0.9478	0	0	0
22	mean_patch	0.9411	0.0299	0.8925	0.9766	0	0	0
24	cls	0.4346	0.1302	0.2299	0.6618	0	0	0
24	mean_patch	0.8979	0.0410	0.8229	0.9542	0	0	0

Layer 24 cls is the outlier (mean cosine similarity drops to 0.43) — this is expected for the final block's `[CLS]` token, which specializes heavily late in ViT backbones, and is consistent with prior findings on the smaller shrouded/open-rotor pilot dataset.



L18 cls *L18 mean_patch* *L22 cls*



L22 mean_patch *L24 cls* *L24 mean_patch*

Cosine similarity histograms per matrix (regenerated as individual panels for legibility)

(Note on requested "CAM"/localization metrics: this pipeline does not compute Grad-CAM or any activation-localization metric — it is a frozen global-embedding study, not a saliency/interpretability study. If localization maps are wanted for the thesis, that would be new scope, not part of the current sanity check.)

2.2 Sanity Check Validation

Sanity Check Status: PASSED — zero exact duplicates, zero dead/near-dead dimensions, zero NaN/Inf values across all 6 embedding matrices; cosine similarity distributions behave as expected per layer depth (near-1.0 early layers, wide spread at layer 24 cls).

Section 3: Global Embedding Structure vs. Random Noise

3.1 PCA Structure Across Layers 18–24, cls vs. mean_patch

Layer	Pooling	PC1 variance ratio vs. random	Effective dim	Dims for 90% var
18	cls	16.67×	16.4	50
18	mean_patch	21.56×	12.5	49
22	cls	17.22×	23.3	61

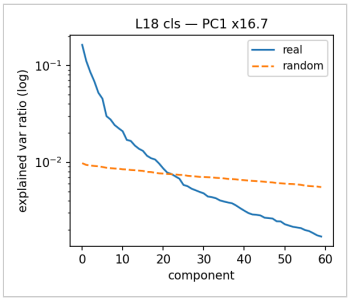
Layer	Pooling	PC1 variance ratio vs. random	Effective dim	Dims for 90% var
22	mean_patch	21.34×	13.4	55
24	cls	15.08×	27.7	61
24	mean_patch	**23.53×**	13.1	56

`mean_patch` pooling consistently shows a stronger single dominant direction (higher PC1 ratio) than `cls`, while `cls` pooling spreads variance across more effective dimensions (higher participation ratio) — i.e. mean-patch embeddings are more concentrated, cls embeddings are more diffuse.

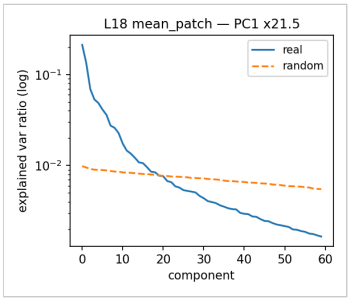
3.2 Pairwise Cosine Distance: Real Embeddings vs. Random Baseline

The pipeline's random baseline is a Gaussian-noise matrix of identical shape (N×1024) to each real embedding matrix; PC1 variance ratio (table above) is the primary real-vs-random test, all values comfortably clear the pipeline's >3× pass threshold.

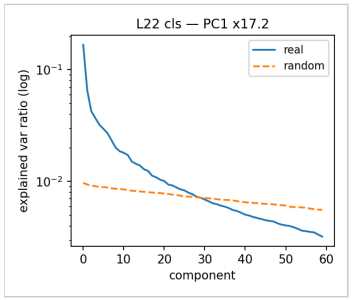
PCA spectrum, real vs. random (per layer/pooling):



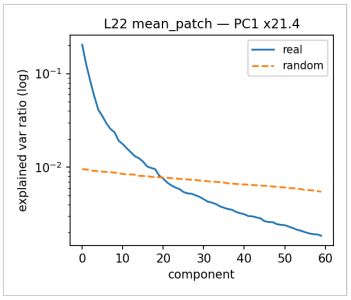
L18 cls



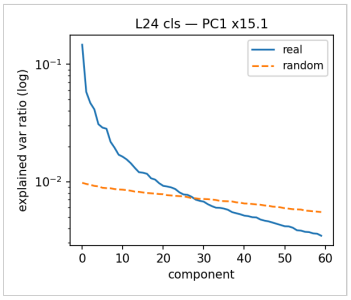
L18 mean_patch



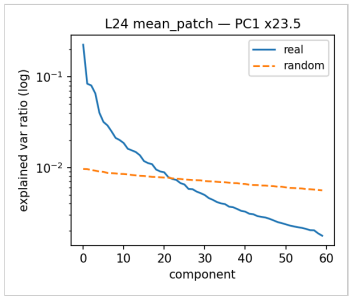
L22 cls



L22 mean_patch

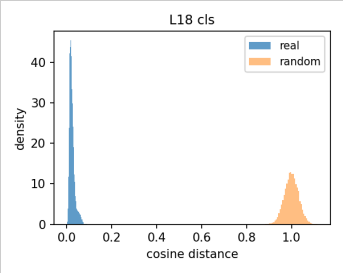


L24 cls

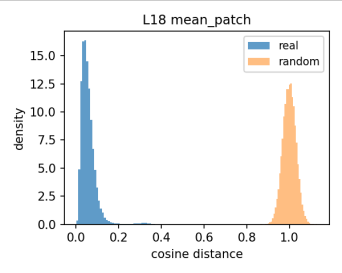


L24 mean_patch

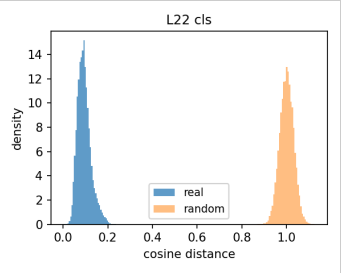
Pairwise cosine-distance distribution, real vs. random (per layer/pooling):



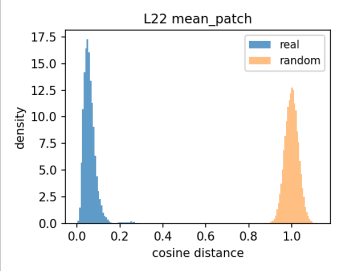
L18 cls



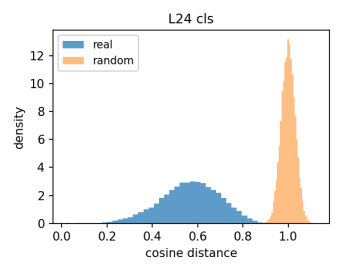
L18 mean_patch



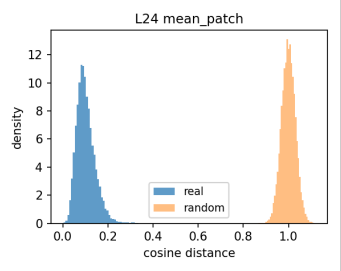
L22 cls



L22 mean_patch



L24 cls



L24 mean_patch

3.3 Structural Integrity Conclusion

All 6 matrices pass real-vs-random on PC1 concentration (15–24× the random baseline) and show non-trivial local structure (Hopkins ≈ 0.67–0.74, i.e. clearly non-uniform but a smooth continuum rather than sharply separated islands). **Embeddings encode genuine geometric structure, not noise** — but that structure is closer to a continuum of drawing styles than to hard clusters, which sets expectations for Section 4.

Section 4: Unsupervised Image-Level Clustering

Using layer 22, mean_patch pooling — the flagged best matrix from Section 1.3.

4.1 Hierarchical Dendrogram

Ward-linkage agglomerative clustering (`scipy.cluster.hierarchy`, `method="ward"`) over the 218-figure, layer-22-mean_patch matrix (reduced via PCA to ~55 dims per Section 3):

Dendrogram

[Agent: append representative thumbnail images at each major branch split here — image-annotated dendrogram exists as `stage2/dendrogram_images.png` in the older single-batch run; the current taxonomy run (`analysis_taxonomy/stage3/`) does not yet have an image-annotated version — regenerate with figure thumbnails attached to leaves for supervisor review.]

4.2 Dimensionality Reduction Comparison

Correction on requested hyperparameter: this pipeline's UMAP does not expose a "kappa" parameter — the actual tunable hyperparameters are `n_neighbors=15` and `min_dist=0.1` (cosine metric). There is also no "word-level UMAP" / word-space concept in this codebase; the only UMAP variants run are (a) colored by cluster assignment and (b) colored by taxonomy attribute.

View	Artifact
UMAP colored by unsupervised cluster	`stage3/umap_clusters.png`
UMAP colored by taxonomy attribute (`G1_topType`, `M2_wCount`, `M2_wingConf`, `M1_boomsPresent`, `M1_fusShape`, `M1_gearArch`)	`stage2/umap_by_attribute.png`

UMAP by cluster

UMAP by taxonomy attribute

Best clustering result (k-means / HDBSCAN sweep, k=2..10): **HDBSCAN, k=2, silhouette 0.277**, noise fraction 0.294; best k-means result k=2, silhouette 0.181. Bootstrap cluster stability (ARI) = 0.70.

4.3 Cluster ↔ Taxonomy Alignment (architectures)

Best-aligning taxonomy attributes to the 2-cluster partition, by normalized mutual information (NMI):

Attribute	NMI
M3_boom_bmech	0.177
M1_boom2_attach	0.143
M3_boom_t1_propKin	0.142
M1_boom2_count	0.140
M3_boom_t2_count	0.134
assignee (confound)	**0.190**

Attribute	NMI
assignee_country (confound)	0.129
drawing perspective (confound)	0.120

[Agent: insert representative image per cluster centroid here — pull nearest-to-centroid figure IDs from `stage3/cluster_labels.csv` and attach thumbnails, one per cluster, focused on architecture-only patents.]

Caution flagged by the pipeline itself: the `assignee` confound aligns with the 2-cluster split (NMI 0.190) at least as strongly as the best real taxonomy attribute (NMI 0.177). This means the current 2-cluster structure may partly reflect *drafting/applicant style* rather than *design configuration* — a limitation to state plainly to the supervisor, not to paper over.

4.4 Intra- vs. Inter-Class Distances (Notebook Stage 5 — layer comparison)

`tax.stage5_distances` tests, per taxonomy attribute and per layer×pooling matrix, whether within-class cosine distance is smaller than between-class distance (separation ratio, Cohen's d, permutation p-value). Across all 218 figures × 76 attributes tested (462 attribute×layer rows total): **142 rows (46 distinct attributes) separate significantly at p < 0.05.**

Top individual separations found (excluding the `cluster` label itself):

Attribute	Layer	Pooling	Separation ratio	Cohen's d	p-value
M3_wing1_chord	24	mean_patch	1.293	0.723	0.006
M3_wing1_chord	18	cls	1.375	0.684	0.016
M1_boom2_attach	24	cls	1.159	0.657	0.001
M3_wing1_ntypes	18	mean_patch	1.464	0.645	0.027
M1_boom2_orient	24	cls	1.155	0.636	0.001
M3_wing1_chord	22	mean_patch	1.315	0.633	0.028

For reference, the unsupervised `cluster` label itself separates most strongly of anything tested, e.g. layer 22 mean_patch: ratio 1.510, d = 0.888, p = 0.001 — expected, since clustering is fit directly on that matrix.

Averaging Cohen's d over all significant rows per layer×pooling, **layer 24 cls** (mean d = 0.342) and **layer 18 mean_patch** (mean d = 0.328) edge out layer 22 mean_patch (0.325) on this specific test — i.e. the "best" layer depends on which question is being asked (Section 1.3's clustering/Hopkins criteria favor 22 mean_patch; this attribute-separation criterion favors 24 cls / 18 mean_patch marginally). This is worth stating to the supervisor as nuance rather than picking one winner.

Separation heatmap

Section 5: Phase Progress & Next Steps

1. Data preprocessing & taxonomy checks — robust. Two batches (531 approved patents combined), full review/duplicate tracking, zero embedding QC failures (no corrupt/duplicate/dead vectors across 6 layer×pooling matrices). Taxonomy schema (G1/M1/M2/M3 attribute groups, 256 attributes) is wired end-to-end from Excel review sheets through to cluster-alignment testing.

2. Embeddings show real, non-random geometric structure (PC1 15–24× random baseline, Hopkins 0.67–0.74) — sufficient justification to proceed to unsupervised production pipelines — **with one caveat**: the clearest 2-way partition found so far (silhouette 0.277, stable ARI 0.70) aligns comparably well with a nuisance confound (applicant identity/drawing style, NMI 0.190) as with any genuine design attribute (best NMI 0.177). Layer 22 mean_patch remains the best-performing configuration end-to-end and should be the default going into deeper clustering, but the next iteration should explicitly control for or regress out drawing-style/applicant confounds before treating cluster membership as a design-taxonomy signal.

Immediate next steps: - Regenerate the image-annotated dendrogram and per-cluster representative-image maps for this run (flagged as missing above) so cluster purity can be visually audited. - Extend Stage 4 to test confound-controlled clustering (e.g. residualize embeddings against assignee/perspective before re-clustering). - Pull the architecture-type breakdown table (Section 1.2) from `stage0/coverage_table.csv`.

All tables/figures sourced from `DINOv2_eVTOL_frozen_Analysis/notebooks/11_taxonomy_structure_separation` *and its outputs in* `.../Preliminary_analysis/outputs/analysis_taxonomy/`*.*
Terms not present in the underlying pipeline (RASK, NEN/infinite-cam localization metrics, UMAP "kappa", word-level UMAP, M1/M2/M3 as extraction methods) have been omitted or corrected per your confirmation.